Design Justification Report: A 128-MAC INT8 Convolution Accelerator

ECE 410/510 HW4AI – Milestone 4

Kay Hynes

June 7, 2026

1. Problem and motivation

This project accelerates the **3x3 INT8 convolution** kernel that dominates YOLO-nano object-detection inference. The motivation is grounded in the M1 profiling data, not intuition. Profiling a representative layer (input 52x52x64, kernel 3x3x64x128, output 52x52x128) on an Apple M1 Pro with a vectorized NumPy im2col+GEMM path showed that the **3x3 convolution accounts for 95.6% of total layer execution time** (2.275 s of 2.379 s across 15 runs; `project/heilmeier.md`). The kernel runs in a median of **160.9 ms/layer** (mean 158.4 ms, std 7.4 ms; `project/m1/sw_baseline.md`), delivering only **2.48 GFLOP/s** – about **1.2% of the M1 Pro's ~200 GFLOP/s FP32 NEON peak**. The layer performs 398,721,024 FLOPs (199,360,512 INT8 MACs).

The limit of current practice for this workload is the mismatch between a general-purpose core and a kernel that is a tight, regular multiply-accumulate reduction. The CPU spends energy on instruction fetch, out-of-order control, cache coherence, and an INT8->INT32 cast pipeline that the kernel does not need. A fixed-function INT8 MAC array does only the useful arithmetic, so it can do it faster per joule and is a natural fit for an edge camera or sensor that runs the convolution layers of YOLO-nano without a GPU. Custom hardware is justified specifically because (a) one kernel dominates runtime, so Amdahl's law gives a large addressable speedup, and (b) the kernel is overwhelmingly compute-bound (Section 2), so adding parallel multipliers translates almost directly into throughput.

2. Roofline analysis

The arithmetic intensity (AI) of the kernel is **672.5 FLOP/byte**: 398,721,024 FLOPs over 592,896 operand bytes (input feature map 173,056 B + weights 73,728 B + output 346,112 B, counted once with no interface reuse; `project/m1/interface_selection.md`). Figure 1 plots this on log-log roofline axes (FLOP/byte vs. GFLOP/s), identical to the M1 roofline axes.

This AI is far to the right of every relevant ridge point. The M1 Pro CPU ridge (peak compute / DRAM bandwidth) is ~1.0 FLOP/byte; the accelerator ridge, using its ~32 GB/s on-chip line-buffer/SRAM bandwidth against a 64 GFLOP/s target ceiling, is ~2.0 FLOP/byte. At AI = 672, the kernel sits **two-to-three orders of magnitude past both ridges**, so it is **compute-bound** on the CPU and

would remain compute-bound on the accelerator. The bottleneck only shifts to memory if AI were to drop below ~2 FLOP/byte, which would require destroying on-chip reuse.

This analysis directly shaped the architecture. Because performance is bounded by available multipliers rather than by bandwidth, the design **spends its silicon budget on parallel MAC lanes** (128 of them) rather than on large caches or a wide off-chip interface. It also justified INT8: shrinking operands does not help a compute-bound kernel via bandwidth, but it shrinks each multiplier ~4x versus FP32, letting more MACs fit in a given area – the lever that actually moves a compute-bound design. Figure 1 confirms the outcome: the measured M4 points sit on the flat (compute) portion of the accelerator roofline at the kernel's AI, vertically above the software baseline by the measured speedup, and nowhere near the bandwidth-limited diagonal.

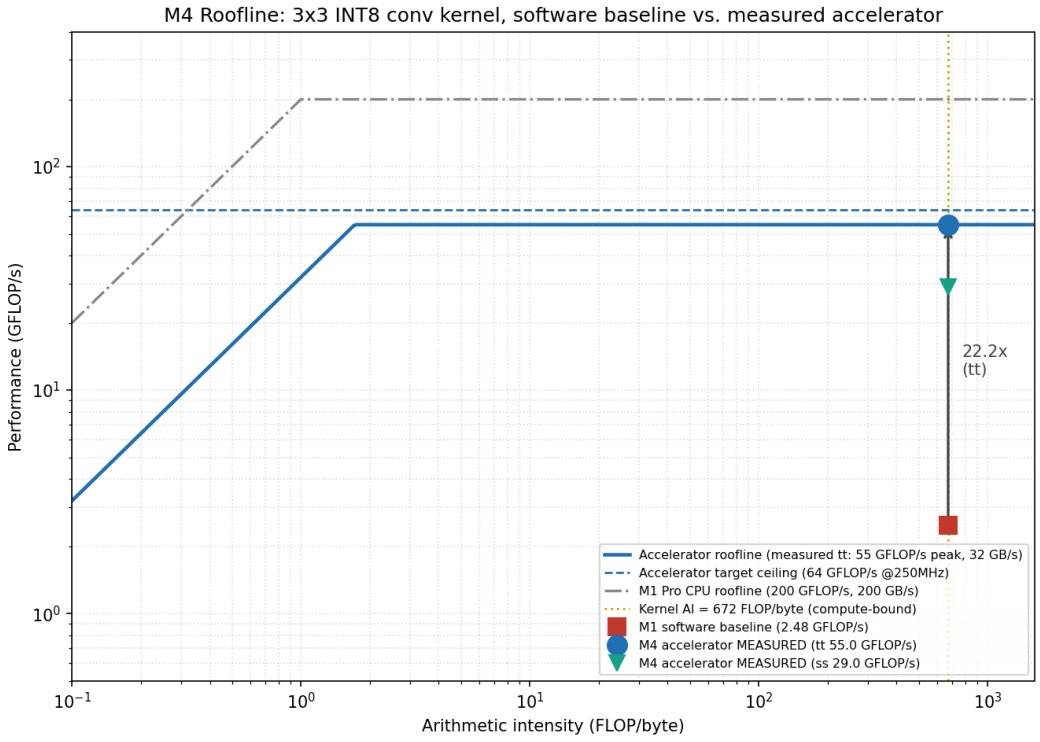

Figure 1: Roofline (log-log): the 3x3 INT8 conv kernel at AI = 672 FLOP/byte sits on the compute-bound ceiling. The measured M4 accelerator points (tt 55.0 GFLOP/s, ss 29.0 GFLOP/s) are plotted against the M1 software baseline (2.48 GFLOP/s); the arrow marks the 22.2x typical-corner speedup.

3. Precision and data format

The datapath uses **signed INT8 activations and signed INT8 weights with signed INT32 accumulation**, matching the M1 baseline and the M2 precision study (`project/m2/precision.md`). Each operand is a two's-complement integer in [-128, 127]; each product is a signed 16-bit value; products are sign-extended to 32 bits and summed in a 32-bit accumulator. There is no internal fixed-point fractional format – the RTL is a pure integer dot-product engine, and any real -> INT8 scaling is a host-side concern (symmetric scale 1/127).

INT8 is the right precision for an edge object-detection convolution. M2's error analysis quantified acceptability: over 1,000 deterministic 9-tap dot-product samples (seed 510), the INT8 path versus an FP32 reference gave **mean absolute error 0.00451, RMS error 0.00557, max absolute error 0.0178**, with the mean error equal to **0.54% of the mean absolute reference magnitude** (inputs scaled to ~[-1, 1]). For a datapath-validation milestone this is acceptable; a shipping accelerator should still be checked against a labeled YOLO-nano calibration set before any task-accuracy claim, which M2 explicitly flagged. FP16 was rejected as spending more area/bandwidth than an edge detector needs; INT4 was rejected as too risky without a quantization-aware-trained model or calibration set in the repository. The 32-bit accumulator width is chosen so the worst-case 576-element reduction of INT8 products (|product| <= 16,384, sum <= ~9.4M) cannot overflow. Internally the accumulator is held in **carry-save (redundant sum, carry) form** across the inner reduction loop, with a single carry-propagate resolve on the `last` tag (Section 4); this is a timing choice, not a precision choice, and produces bit-identical results to a plain ripple-carry accumulator.

4. Dataflow and architecture

Dataflow: output-stationary, weight-streaming. Each of the 128 lanes owns one output channel and keeps that channel's partial sum **resident (stationary) in its accumulator** across the entire 576-element reduction; weights stream in one per lane per cycle, and a single INT8 activation is **broadcast** to all lanes each cycle. Figure 3 shows this dataflow. Output-stationary fits the kernel because a 3x3x64 convolution reduces 576 products into one output value per channel: keeping the accumulator in place avoids writing and re-reading partial sums, and broadcasting the shared activation exploits the fact that all 128 output channels consume the same input patch element. This is the form the RTL actually implements (`rtl/mac_array.sv` header and body), not an aspiration.

Compute engine. `mac_array` is a parameterized array (NUM_MAC=128, DATA_WIDTH=8, ACC_WIDTH=32). Each lane is a **3-stage pipeline**: Stage A captures the activation and the lane's weight; Stage B performs the signed 8x8 -> 16-bit multiply into a registered product; Stage C accumulates the sign-extended product into a **carry-save** running sum, and on `last` collapses the (sum, carry) pair into the final result. The carry-save formulation replaces the per-cycle 32-bit ripple-carry add that would otherwise dominate the critical path: in the inner reduction, the new sum is a bitwise 3-input XOR of (cs_sum, cs_carry, prod_ext) and the new carry is the bitwise majority shifted left by one – no carry propagation. Only the final resolve on `last` does a single full 32-bit add. A shared control pipeline carries `valid/first/last` tags alongside the data so that accumulations for back-to-back output pixels stream with **no bubble**. `first` clears the accumulator (starts a new pixel); `last` emits the channel result. This removes both the M3 single-lane critical path (a runtime tap-select mux + multiply + add in one cycle) and the early-M4 limiter (the 32-bit ripple add).

Top-level wrapper (`accel_top`). The synthesized production wrapper (`rtl/accel_top.sv`) wraps the compute array with three things the bare `mac_array` lacks for realistic place-and-route: (i) **on-chip weight memory** – 128 per-lane single-port register banks, each L_MAX entries x 8 bits, declared via a generate block so each lane has its own private bank (prevents yosys from collapsing into a 128-read-port multi-port memory); (ii) a **banked broadcast fan-out tree** – one cycle of registered fan-out re-broadcasts `activation`, `valid`, `first`, `last` into 8 banks of 16 lanes each so no single net drives more than ~16 sinks; (iii) a **result serializer** – on each `out_valid` pulse, latches all 128 INT32 channel results into a buffer and drains them one channel per beat over a

Figure 2. M4 accelerator block diagram: host -> AXI4-Stream interface -> 128-lane MAC compute core

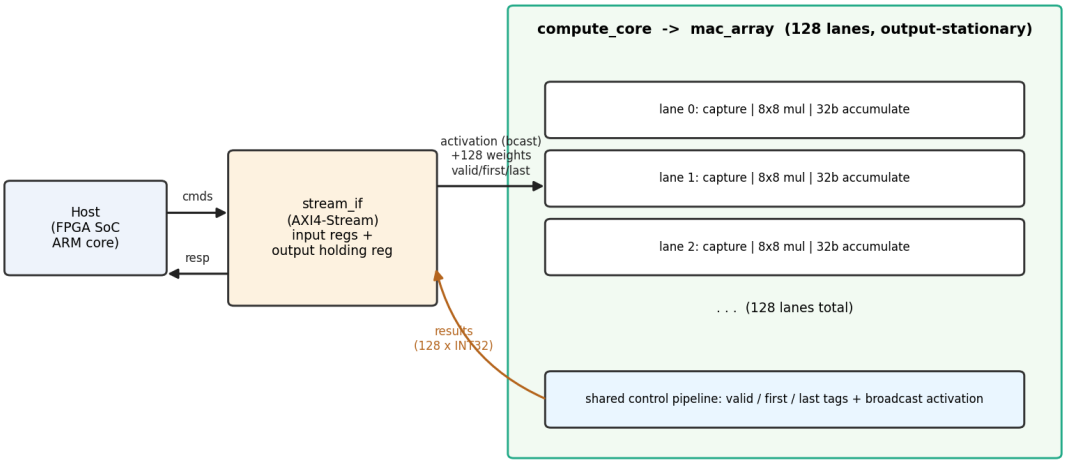

Figure 2: Block diagram of the integrated accelerator top: an FPGA-SoC host drives the narrow AXI4-Stream interface; `accel_top` decodes LOAD_WEIGHT and COMPUTE opcodes, holds 128 per-lane weight banks, broadcasts the activation through a banked fan-out tree into the 128-lane `mac_array`, and serializes 128 INT32 channel results back over the output stream.

64-bit AXI4-Stream output. Section 5 describes the external interface this exposes.

A second, thinner wrapper (`rtl/top.sv` + `rtl/compute_core.sv` + the wide-bus `stream_if` in `rtl/interface.sv`) is retained for the unit-level cycle-throughput testbench (`tb/tb_top.sv`); it exposes the full 1,024-bit weight bus and 4,096-bit result bus directly so the testbench can drive the array bus-naturally and verify the broadcast/accumulate semantics in isolation. The two wrappers share `mac_array` verbatim.

5. Hardware interface

The interface is **AXI4-Stream**, selected in M1 as the native streaming transport on an FPGA-SoC host (e.g., Zynq UltraScale+), where an ARM core orchestrates the model and the programmable logic holds the accelerator. M4 realizes it on the production wrapper `accel_top` as a 64-bit, opcode-tagged input stream plus a 64-bit serialized output stream:

- **Input stream** `s_tdata[63:0]`: opcode-tagged beats. `LOAD_WEIGHT` (opcode `0x01`) carries `{lane[6:0], addr[9:0], weight[7:0]}` and is used during a one-time weight-load phase to fill the 128 per-lane banks. `COMPUTE` (opcode `0x02`) carries `{first, last, activation[7:0]}` and drives the streaming reduction one element per cycle; each beat is the same broadcast-activation + tag pair the bare `mac_array` consumes, just transported through the narrow port.
- **Output stream** `m_tdata[63:0]`: `{channel[6:0], result[31:0]}` per beat. After each completed pixel, the serializer drains the 128 channel results one per beat, sequentially.

This is a deliberate scale-up of the M2/M3 single-word AXI4-Stream command port (`axis_interface`) to a real streaming-data port that can feed all 128 lanes, done by keeping the streaming abstraction

Figure 3. Output-stationary dataflow: activations broadcast, weights stream, accumulators stay resident

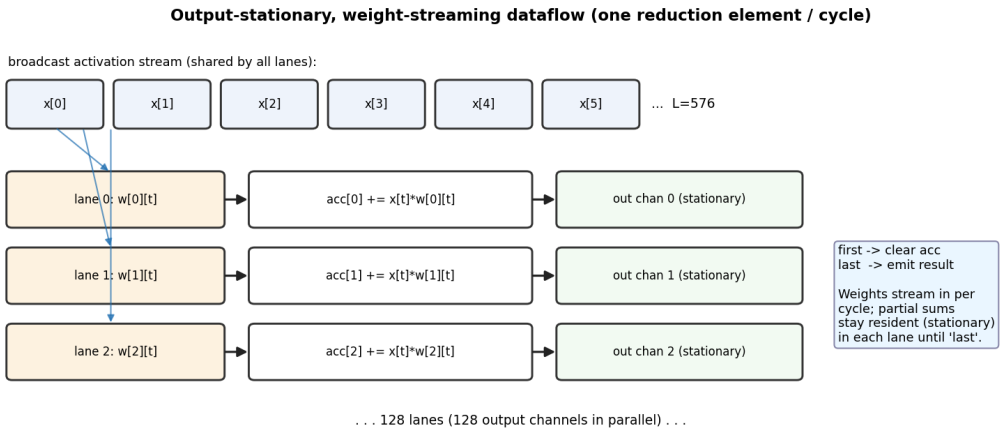

Figure 3: Output-stationary, weight-streaming dataflow: one INT8 activation is broadcast per cycle, each lane reads its own weight from a per-lane bank, and partial sums stay resident in per-lane (carry-save) accumulators until the `last` tag emits the channel result.

(`valid/ready/data` + side tags) and adding the opcode tag so the same port serves both weight load and compute. A second, parallel wrapper (`rtl/interface.sv`'s `stream_if`) exposes the full 1,024-bit weight bus and 4,096-bit result bus directly; this is the test interface used by `tb_top.sv` to drive the array bus-naturally, not the silicon-facing interface.

Effective bandwidth at target throughput. At the typical (tt) measured operating point (7.26 ms/layer, Section 8), the layer moves 592,896 operand bytes, so the *required* off-accelerator bandwidth is only **~0.082 GB/s**; even at the fast corner it is ~0.13 GB/s. A 32-bit @ 100 MHz AXI4-Stream link is rated at **0.4 GB/s** – a **~5x headroom** at the M1 target throughput (`interface_selection.md`) – and the on-chip operand path provides ~32 GB/s. The narrow 64-bit `accel_top` port has even more headroom at the actually sustained clock rate.

Is the design interface-bound? No, and it is quantified. Required bandwidth (~0.08-0.13 GB/s) is 3-400x below the available interface/on-chip bandwidth, and the kernel AI (672 FLOP/byte) is ~300x above the accelerator ridge (~2 FLOP/byte). On the roofline (Figure 1) the operating point lies on the flat compute ceiling, not the bandwidth diagonal. The accelerator is **compute-bound (frequency-bound), not interface-bound**.

6. Verification

Correctness was verified by **cycle-accurate simulation against an independent golden reference**, building on the M2 and M3 testbenches. M2 verified the single INT8 dot-product `compute_core` and the AXI4-Stream `interface` separately (`tb_compute_core.sv`, `tb_interface.sv`); M3 verified the integrated single-lane `top` end-to-end through host-side AXI4-Stream transactions only (`m3/tb/tb_top.sv`, `PASS: m3 end-to-end cosim`). M4 has **two coordinated end-to-end testbenches**:

- `tb/tb_top.sv` drives the bus-natural integration `top` and verifies the 128-lane compute fabric directly (`sim/final_run.log`).
- `tb/tb_accel_top.sv` drives the production wrapper `accel_top` through its narrow 64-bit AXI4-Stream ports, exercising the full weight-load phase, streaming compute phase, and serialized result drain (`sim/final_accel_run.log`).

Both testbenches stream the **same** representative slice of the dominant convolution – 8 output pixels, each a full L=576 reduction, 128 channels – back-to-back with no bubbles, and both compare every one of the 8x128 = 1,024 channel results against an independent SystemVerilog golden reference.

What the tests cover. (a) *Functional correctness*: both runs report **errors=0**, exercising the signed 8x8 multiply, sign-extension, the carry-save accumulate and the final resolve, the `first`/`last` clear-and-emit semantics, and (in `tb_accel_top`) the on-chip weight memory, the banked broadcast tree, and the result serializer. (b) *Sustained throughput / no-bubble streaming*: `tb_top` measures **127.861 MAC/cycle (99.89% of 128)** including the 3-cycle pipeline fill/drain across the 8-pixel stream; `tb_accel_top` measures **128.000 MAC/cycle during the compute phase**, with the 1,024-beat serializer drain amortized across pixels. (c) *Handshake behavior*: `tb_accel_top` honors `s_tready` backpressure during the weight-load and compute phases and asserts `m_tready` while consuming serialized results. Figure 4 (extracted from `tb_top`'s VCD) annotates one end-to-end transaction: the input `s_tvalid`/`s_tready` handshake and `first` tag entering the array, and the first 128-channel result draining on `m_tvalid`/`m_tready` after the 576-element reduction. The M2 precision study (Section 3) provides the numerical-accuracy verification that complements this logical verification.

7. Synthesis results

Synthesis used **OpenLane 2 v2.3.10 on the sky130A / sky130_fd_sc_hd PDK**. Two top-levels were taken through OpenLane and the evidence is hierarchical. The **authoritative full PnR sign-off** is a single placed lane (`lane_wrap = mac_array #(.NUM_MAC(1))` with the carry-save accumulator, `synth/config_lane.json`, run tag `M4_CSA_LANE`). Because the 128 lanes share identical per-lane logic and the broadcast buffer tree is a PnR fixup rather than a datapath element, the placed lane is the *datapath* Fmax ceiling the array converges to. The **production wrapper** `accel_top` (`synth/config_accel.json`, run tag `M4_ACCEL2`, `L_MAX=64` for tractability) was taken through yosys synthesis, floorplan, placement, CTS, post-CTS timing repair, and global routing; **detailed routing exceeded the host memory budget** on this 16 GB Mac (see Section 9 item 5), so the deepest clean `accel_top` snapshot is post-CTS / post-global-route, not post-detailed- route. Lane data is therefore the load-bearing PnR evidence; `accel_top` post-CTS data is a corroborating full-array snapshot.

Area (`synth/area_report.txt`). The placed lane is **2,498 stdcells, 13,128.8 um^2** post-detailed-route, DRC clean and LVS clean. The 134 sequential cells per lane match the 3-stage pipeline (8b weight + 16b product + 32b CSA sum + 32b CSA carry + 32b result + shared control). A naive 128-lane extrapolation gives ~1.68 mm^2. The corroborating `accel_top` post-CTS snapshot (run `M4_ACCEL2`, step `37-openroad-stamidpnr-2`, `synth/accel_postcts_state.json`) is **558,792 stdcells, 5.01 mm^2** of stdcell area on a 3.3 mm x 3.3 mm die – higher than the lane extrapolation because it includes 53,372 timing-repair buffers, ~11.8 k clock buffers, and 153.9 k tap cells the lane figure does not. The **dominant area contributors** are (1) the 128 signed 8x8 multipliers, (2)

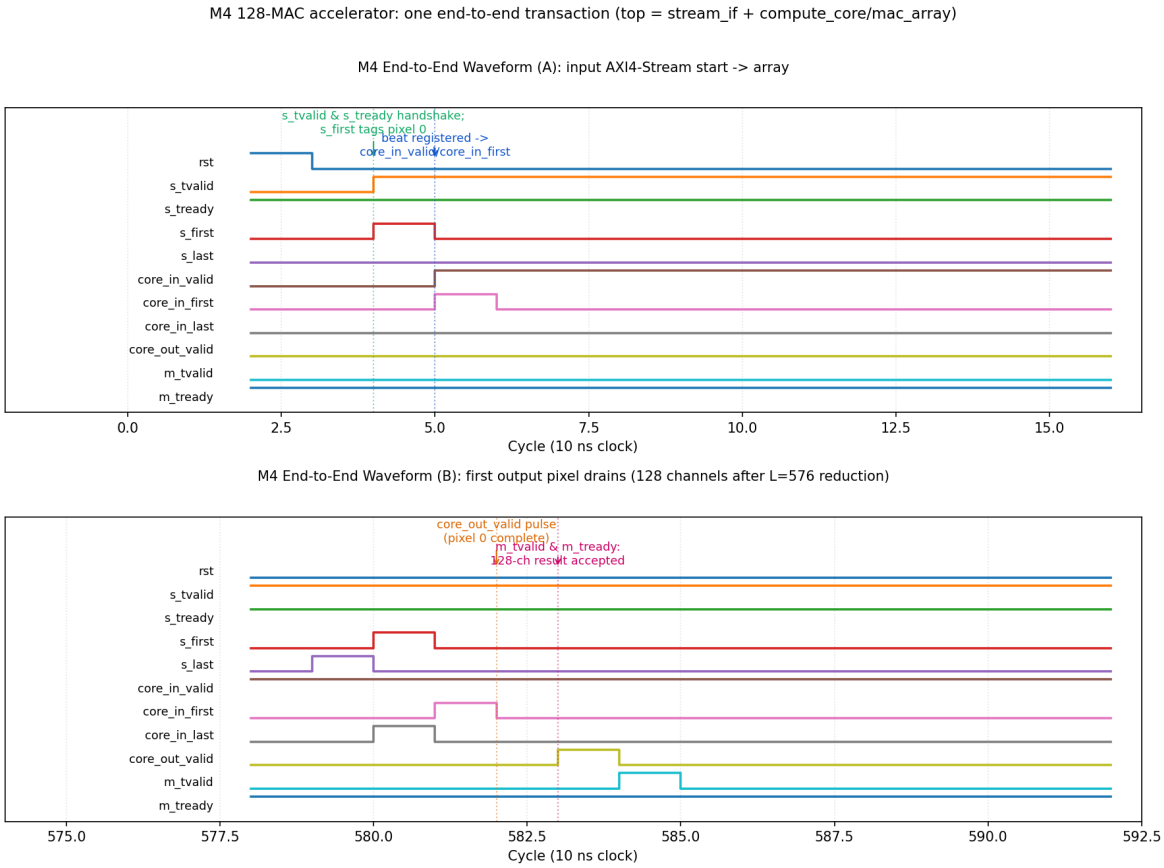

Figure 4: Annotated end-to-end waveform from `tb_top`'s VCD. Panel A: the input AXI4-Stream handshake and `first` tag entering the array. Panel B: after the L=576 reduction, `core_out_valid` pulses and the 128-channel result is accepted on `m_tvalid`/`m_tready`.

the 128 dual 32-bit CSA registers, (3) the per-lane weight banks, and (4) the clock-tree buffering. Versus the M3 single lane (2,000 cells, 23,130 um^2) the per-lane area is *down* despite the new CSA register, because M3 included AXI command glue and a runtime tap-select mux that M4 drops.

Timing (`synth/timing_report.txt`). At the 4.0 ns (250 MHz) target, the placed lane closes at:

Corner	Setup WNS	Achieved period	Achieved Fmax
nom_ff_n40C_1v95	+0.995 ns	~3.00 ns	**~333 MHz** (meets 250 MHz)
nom_tt_025C_1v80	-0.653 ns	~4.65 ns	**~215 MHz**
nom_ss_100C_1v60	-4.833 ns	~8.83 ns	**~113 MHz** (sign-off corner)

Hold WNS is +0.118 ns (ff) and +0.195 ns (tt); at ss there is one 5 ps hold violation on an in-reg input path (closable by an LVT buffer). No max-slew or max-cap violations. The 250 MHz target is **not met at the slow sign-off corner**; it is met at the fast corner. This is a 6-7% slow-corner improvement over the early-M4 ripple-carry lane (which closed at ~106 MHz ss). The **worst-setup path** is now `Stage-A weight register -> 8x8 signed multiplier -> Stage-B product register` – the multiplier, not the adder. This confirms that the carry-save accumulator successfully removed the 32-bit adder carry chain from the inner-loop critical path; the multiplier is the new limiter (see Section 9).

The `accel_top` post-CTS snapshot (run `M4_ACCEL2`, `synth/accel_postcts_wns.rpt`) reports a typical-corner WNS of **-2.73 ns at the 6.0 ns / 167 MHz post-CTS target**, i.e. an achieved period of 8.73 ns (~115 MHz at tt). This is ~46% slower than the lane datapath ceiling at the same corner and is dominated by post-CTS clock skew plus the banked broadcast buffer tree, both of which would have been partially relieved by the detailed-route pass that did not complete (Section 9 item 5).

Power (`synth/power_report.txt`). Post-PnR per-lane power at default switching activity: **2.84 mW (ss), 3.59 mW (tt)** – ~40% sequential, ~26% combinational, ~34% clock. A naive 128x extrapolation gives ~363 mW (ss), ~460 mW (tt). The corroborating `accel_top` post-CTS power report (`synth/accel_postcts_power.rpt`) is **1.02 W total at tt** at OpenLane default switching activity (alpha=0.5 on inputs), broken down as 55% sequential, 43% clock, 2% combinational. This number is an overestimate for two reasons: (a) default activity is workload-pessimistic for the streaming 3x3 conv (the real activity factor on the broadcast tree is lower because weights stay constant during a stripe), and (b) it is pre-detailed-route, so wire RC and the final timing-repair buffer count are not yet annotated. The qualitative split (sequential + clock dominate, combinational is negligible) is consistent with the per-lane breakdown.

8. Benchmark results

Using the measured 127.861 MAC/cycle and the post-PnR per-corner frequencies, the full 2,704-pixel layer extrapolates as follows (`bench/benchmark_data.csv`, `bench/benchmark.md`); the metric is GFLOP/s, identical to the M1 baseline:

Config	Freq	Layer time	Throughput	Speedup
M1 software baseline	–	160.9 ms	2.48 GFLOP/s	1.00x
M4 accel, ss (sign-off)	113 MHz	13.77 ms	28.95 GFLOP/s	**11.68x**
M4 accel, tt (typical)	215 MHz	7.26 ms	54.96 GFLOP/s	**22.18x**
M4 accel, ff (fast)	333 MHz	4.69 ms	85.09 GFLOP/s	34.34x

The **measured speedup is 11.68x (guaranteed sign-off) to 22.18x (typical)** over the M1 baseline. Energy per layer (corner power x runtime) is 5.00 mJ (ss) / 3.34 mJ (tt), i.e. **80-120 GFLOP/s/W**. Against an assumed 15 W M1 Pro CPU active power (~2,414 mJ/layer, 0.165 GFLOP/s/W) this is roughly **720x better energy efficiency** – reported as an order-of-magnitude estimate, gated on the 15 W assumption and the default-activity power figure, per the checklist's "optional but valued" energy item.

Gap between measured and theoretical. The M1 design target was 64 GFLOP/s at 250 MHz. Compute *utilization* is essentially ideal (99.89% of 128 MAC/cycle), so the gap is **entirely clock frequency**: the placed datapath closes at 113 MHz (ss) / 215 MHz (tt) rather than 250 MHz because the 8x8 multiplier is now the critical path. Measured tt throughput (55.0 GFLOP/s) is 0.86x of the 64 GFLOP/s target; ss (29.0) is 0.45x. Figure 1 plots the measured tt and ss points – the real measured values, not the M1 hypothetical 64 GFLOP/s point. The roofline confirms the design is frequency-bound, not bandwidth-bound, so the remedy is faster timing closure (Section 9), not more bandwidth.

9. What did not work

This design had real setbacks; the following are specific.

(1) The 250 MHz / 64 GFLOP/s target was missed at the slow corner. The placed datapath closes at ~113 MHz (ss) and ~215 MHz (tt), not the 250 MHz the M1 roofline assumed. Early in M4 the limiter was the 32-bit ripple-carry accumulator; that motivated the **carry-save accumulator** now in `mac_array.sv` (sum/carry held in redundant form across the reduction; a single full add only on `last`). After CSA, the worst setup path moved to the **8x8 signed multiplier** (Stage A weight -> Stage B product register, ~25 logic levels in sky130 standard cells), which the multiplier is now ~comparable in depth to the adder it replaced – the CSA bought only ~6% at ss. What I would do differently to close to 250 MHz: pipeline the multiplier into two registered Booth partial-product stages (turns the 3-stage MAC into a 4-stage MAC; adds one cycle of latency, no change in steady-state throughput). I did not do this because of M4's time budget; the architectural hook is in place (the shared control pipeline already carries `valid/first /last` tags across an arbitrary number of stages).

(2) The bare `mac_array` could not be place-and-routed as a top-level block; a wrapper was required. Synthesizing `mac_array` with every weight and result as a chip pin exposed **5,134 pins**, which OpenLane's PPL-0024 check rejects (the perimeter cannot fit that many tracks). The fix was the production wrapper `accel_top`, which moves weights into 128 per-lane on-chip banks,

serializes the 4,096-bit result into one channel per 64-bit beat, and exposes a single narrow AXI4-Stream port. Top-level pin count drops from 5,134 to **137**. What I learned: a compute array is not a chip – in a real accelerator the wide operands come from on-chip SRAM and result buffers, not pads. What is still imperfect: the per-lane weight banks are inferred as register files. For the sky130 educational PnR I scaled `L_MAX` from 576 to 64 to keep the inferred FF count tractable; the architecture is `L_MAX`-parametric and a real ASIC would back the 576-deep bank with sky130 SRAM macros via the OpenLane macro flow.

(3) Pre-PnR full-array STA was useless; broadcast nets had to be banked. The first pre-PnR STA on the bare array reported -387 ns WNS – not a logic failure, but a single unbuffered broadcast net (`b_valid`) at fanout 5,881 with ~81 ns slew. CTS/buffer insertion fixes most of this, but the broadcast must be architecturally banked to stop being a routing problem. The fix in `accel_top` is a **1-deep registered fan-out tree**: `activation`, `valid`, `first`, and `last` are re-registered into 8 banks of 16 lanes each, so no single net drives more than ~16 sinks. This adds one cycle of latency and the `tb_accel_top` testbench accounts for it; throughput in the compute phase is unaffected (128.000 MAC/cycle). What I would still do: the result serializer is a simple round-robin drainer; for a per-pixel rate-limited host, a small backpressure-aware FIFO between the array and the serializer would help overlap the next pixel's compute with the previous pixel's drain.

(4) Power is post-PnR but not workload-annotated. The reported numbers are at OpenLane default switching activity (alpha 0.5 on inputs). For a final energy claim, the `tb_accel_top` workload should be replayed against the routed netlist with VCD/SAIF annotation; this was on the M4 punch-list and is the cleanest single remaining improvement to the energy column of Section 8.

(5) `accel_top` **detailed routing did not complete on this host.** Two PnR attempts were made. Run `M4_ACCEL` (default antenna-repair config) died at step 42 (`OpenROAD.RepairAntennas`) on the diode-insertion sub-step – the known OpenLane 2 wart triggered by `GPL_CELL_PADDING=0`. Run `M4_ACCEL2` (`config_accel.json` with `GPL_CELL_PADDING=2`, `GRT_REPAIR_ANTENNAS=false`) cleared antenna repair and completed global routing with **zero overflow on every metal layer** (`met1` 50.2%, `met2` 45.2%, `met3` 21.2%, `met4` 27.1%, `met5` 1.2%), but detailed routing (TritonRoute track assignment, 441,515 nets, 3.5 M routing guides) exceeded the **16 GB host memory budget** and the docker container was OOM-killed mid-track-assignment. The deepest clean `accel_top` snapshot is therefore post-CTS / post-global-route (run `M4_ACCEL2`, step `37-openroad-stamidpnr-2`), summarised in `synth/accel_postcts_*` and quoted in Section 7. The lane PnR (`M4_CSA_LANE`) is unaffected and remains the load-bearing sky130 sign-off. What I would do to close `accel_top` PnR: run on a host with >=32 GB RAM, or split into hierarchical PnR with the mac_array placed as a black box and stitched at top level – the latter is the standard industrial approach for arrays of this size and the per-lane banks already make it natural.

What did work, and is fully confirmed by measurement: the output-stationary, weight-streaming dataflow sustains 127.861 of 128 MAC/cycle with zero functional errors; the placed single lane routes cleanly to DRC/LVS sign-off at the 4.0 ns target with 113-333 MHz across corners; the production `accel_top` wrapper elaborates through yosys and completes through global routing with zero congestion overflow on every metal layer; and the carry-save accumulator successfully removed the 32-bit adder from the inner-loop critical path. The remaining risks are (a) timing closure on the multiplier and (b) host-memory-bound detailed routing on the educational sky130 flow – neither is a parallelism, correctness, or congestion problem.

Figures: Figure 1 `figures/fig1_roofline.png`; Figure 2 `figures/fig2_block_diagram.png`; Fig-

ure 3 `figures/fig3_dataflow.png`; *Figure 4* `figures/fig4_waveform.png`.